

COMP 452 Assignment 2, Part 1 (A*) Description File

Bryce Dixon (3397649) Jan 2026

TABLE OF CONTENTS

- PART 1: Logic and Game Design - Page 1
- PART 2: Game Compilation Instructions - Page 6
- PART 3: Game Play Instructions - Page 7
- PART 4: Current Game Bugs/ Problems - Page 8

PART 1: Logic and Game Design

DESIGN GOAL:

I wanted my program to be intuitive and clear to use, while utilizing colours to enhance, and not distract from, the design and visuals. This allows for a clear demonstration of how the A* algorithm is working as it searches a grid for a path.

CLASS STRUCTURE OVERVIEW

- Main
 - GridCellSelection
 - AStarGridNode
 - AStarMain
 - AStarListManager
 - AntObject

GAME LOGIC OVERVIEW

GRID CREATION LOGIC

- INITIALIZE GRID:
 - The game loop is paused (using a boolean) during the grid initialization and filling.
 - For creating the grid, the program first creates a GridPane object and a GridCellSelection 2D array. GridCellSelection is an object which each instance represents one cell of the grid.
 - The program draws a 16x16 grid by populating the GridPane object with GridCellSelection objects, and also adds this object to the 2D array.
 - the GridPane object is used to draw the grid on the game scene, while the 2D array stores the grid for the A* algorithm to use.
 - The terrain type buttons are created on the right hand side as a ToggleGroup of ToggleButtons. This allows the buttons to be selected as a mutually exclusive group (i.e. only one at a time). When a button is selected, the program stores that button's terrain type as the currently selected terrain.
 - Each GridCellSelection object in the visible grid is made clickable (using an event handler), where when a grid cell is clicked, that GridCellSelection object will have its terrainType and cost set to the program's currently selected terrain type (i.e. the button that the user has selected).

- USER GRID ENTRY
 - As described above, the user selects the button for the terrain they want to use, and then clicks the grid cell to place that terrain in that cell.
 - When the user has selected a start or goal node, the program stores that start or goal node.
 - Once completed, they click 'Run A*'. The program first checks if the user has input a start and goal node. If they have not, the program returns to its grid data collection state and indicates this to the user. If these do exist, the program disables all buttons and the grid, adds the start node to the open list, and begins the game loop to run A*.
- A* RUNNING LOGIC
 - One each game loop iteration, the program does the following:
 - Check if the open list is empty.
 - If it is, there is no path, indicate this to the user and stop the loop.
 - If it is not empty, get the next open list node.
 - If this node is the goal node, we have found the path.
 - Stop the game loop.
 - Place the path in a stack (working from goal to start using child>parent relationships).
 - Display the path on the grid by popping the path off the stack from start to goal.
 - Add the path to an ArrayList while doing this (the arraylist will be ordered from start to goal).
 - Initiate the Ant object to move along the path given by the ArrayList.
 - If it is not the goal, get this node's neighbours:
 - Only implementing 4 direction travel (left, right, up, down), no diagonal movement.
 - Check for edge conditions for nodes on the edge of the map to avoid index out of bounds errors.
 - Remove any obstacles from the neighbours list.
 - Remove the current parent node from the neighbours list (as to not back track).
 - Check if the neighbours exist in the open or closed lists
 - If in the open list:
 - Check if the g-value of the node in the open list is greater than that of the newly found g-value.
 - If it is, update the g-value, and new parent of the node in the open list.
 - Remove this node from the neighbours list.
 - If in the closed list:
 - Check if the g-value of the node in the closed list is greater than that of the newly found g-value.

- If it is, update the g-value, and new parent of the node in the closed list.
 - Remove this node from the neighbours list.
 - Update the h- and g-value and parent node (i.e. the current node being expanded) of the remaining neighbours in the list.
 - Only h and g values are stored, as my AStarNode has a GetF() function that returns g+h.
 - Add the remaining neighbours to the open list.
 - Add the expanded node to the closed list.
 - This continues until the **open list is empty** or **the goal node is found**.
- A* ADDITIONAL BACKEND LOGIC
 - The output of the A* search is slowed so the user can more easily see the search in progress. This is done by:
 - Taking a "stepInterval", which is a set amount of time I wanted to pass between each frame.
 - Then the loop keeps track of when the last frame ran.
 - If the amount of time between the current time and the last time a frame ran is greater than the step interval, then we run another frame and set the frame time to now.
 - The heuristic values are determined using manhattan distance.
 - My A* algorithm utilized a list manager that handles all interactions with the open and closed lists.
 - I stored 2 open lists (one is a PriorityQueue and one is a HashSet) as the priority queue is efficient for getting the next next node based on F value (using a custom Comparator), but HashSet is much quicker to determine if a node is within the open list. This was done for efficiency.
 - My closed list is a hashset, as it is efficient for using .contains() and remove() as well and I don't need priority.

ANT OBJECT LOGIC

- The ant object is simply a black circle for this part of the assignment.
- This object takes the found path as an ArrayList of grid cells (from index:0 = start, to index:n = end), and follows it in the following way:
 - The ant object sets both its current row and column, and target row and column values to the start column. It then enters its update loop, which iterates as follows:
 - Get the distance between the target location and the ant's current location (using the row and column values as a and b in the pythagorean theorem to solve for c).
 - If the distance is less than 1 pixel (1 was chosen as values higher than this made the movement jittery):

- Set the ant's current location to the center of the target location (i.e. the center of the target grid cell). This is done to avoid drift, which was initially an issue.
- If we are at the goal node:
 - End the game loop.
- If we are not at the goal, set the next target cell to the center of the next node in the path.
- If the distance to the target cell is greater than 1:
 - We normalize our speed in both the row and column directions.
 - We modify each of these speeds relative to the deltaTime value (so that they are accurate velocities).
 - deltaTime is calculated in the main game loop, and is simply a measure (in seconds) of how long the last frame was.
 - This was the speeds we add in the row and column directions are moving relative the the device's frame rate.
 - These speeds are then used to nudge the ant's current row and column values (using +=).
 - The ant's circle object is then drawn to center at those new row and column values so this change is visible on the screen.

GAME END

- The game and animation ends once the Ant object has traversed the path that A* has found, and the ant has made it to the goal node of the path.

ERROR DETECTION IMPLEMENTED

- NO START OR GOAL NODE ENTERED
 - If the user does not place a start or goal node in the grid, the output text (Label 'E' in Image 3 below) will indicate that these need to be in place for the algorithm to run (image 1). The user is able to modify the grid to place these in.

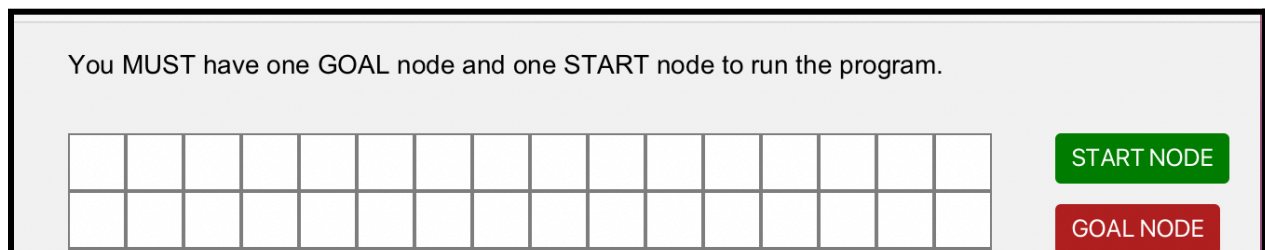


IMAGE 1: No start or goal node entered

- REPLACING START OR GOAL NODES
 - Since the program stores a reference to the goal and start nodes for use and uses this to detect if goal and start nodes have been entered, the program needed to detect if one of these nodes was replaced with a different terrain type.

- If it is replaced, the program needs to replace the reference to this with null to indicate it no longer exists. This was done within the event handler of each cell. This handles all cases:
 - If a start cell is replaced with: goal, swamp, grass, obstacle, or open terrain.
 - If a goal cell is replaced with: start, swamp, grass, obstacle, or open terrain.
- NO PATH EXISTS
 - If there is no path from the start node to the goal node, the system will detect and indicate this (image 2).

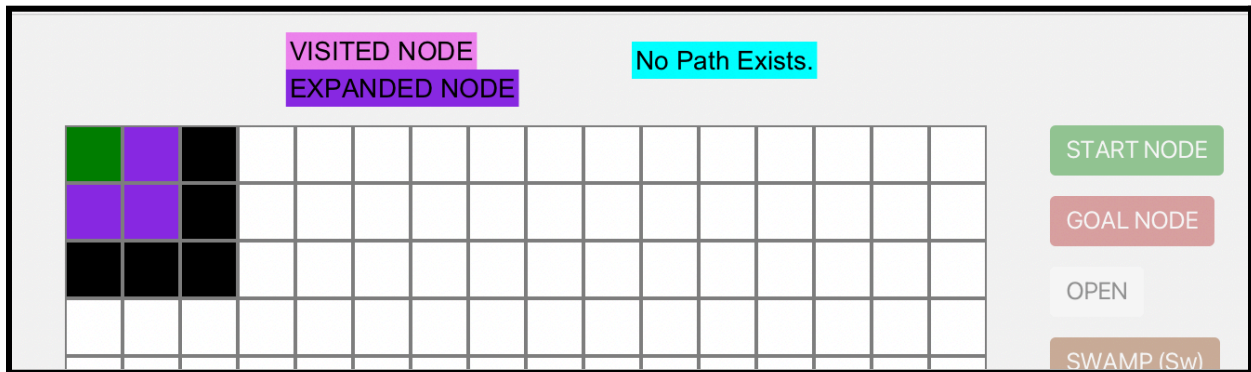


IMAGE 2: No path exists output

- NO MODIFICATION ONCE A* IS RUNNING
 - Once the user enters the grid data and clicks 'RUN A*' the grid and all buttons are disabled so that they cannot modify it once it is run. This can be seen on the right hand side of image 2 above where the buttons are shown faded out.

PART 2: Game Compilation Instructions

IF YOU ARE USING JAVA 8:

- Since JavaFX is included with Java8, you can simply navigate to the directory where you have saved the jar file and run:
 - `java -jar A2P1_452_Dixon_3397649.jar`

IF YOU ARE USING JAVA 11 and HIGHER:

- This project was developed and tested on a MacOS device using Java 17, meaning I needed to download and install JavaFX. I used JavaFX version 21.0.9.
- To run the .jar file, save the file to your device in a known location.
- Ensure JavaFX is downloaded, and find the path to the JavaFX 'lib' folder.
- In your command line interface (CLI), Navigate to the location where you saved the .jar file.
 - Now we tell java what libraries to include from JavaFX, as well as where to find them (i.e. the path to "lib" mentioned above), and then run the program.
 - An example of this is:

```
desktop % java \  
--module-path /Users/bd/Desktop/AU/COMP_452/javafx/lib \  
--add-modules javafx.controls,javafx.graphics \  
-jar A2P1_452_Dixon_3397649.jar
```
 - Where the file (A2P1_452_Dixon_3397649.jar) is stored on on my desktop, the path to my javaFX library is
`/Users/bd/Desktop/AU/COMP_452/javafx/lib`, and we are using the modules `javafx.controls,javafx.graphics` from the JavaFX library to run the program.

PART 3: Game Play Instructions

- Referring to the image below:
 - A. These are the buttons used to select which terrain you want to place.
 - You select the desired terrain button and then click any cell in the grid to place that terrain type.
 - B. This is the grid, all cells are selectable as described above.
 - C. The text at the bottom outlines the costs for each of the terrain types for reference.
 - D. This is the run button that is pressed to run the A* algorithm on the created grid.
 - E. This is output text which changes throughout the program to keep the user updated.

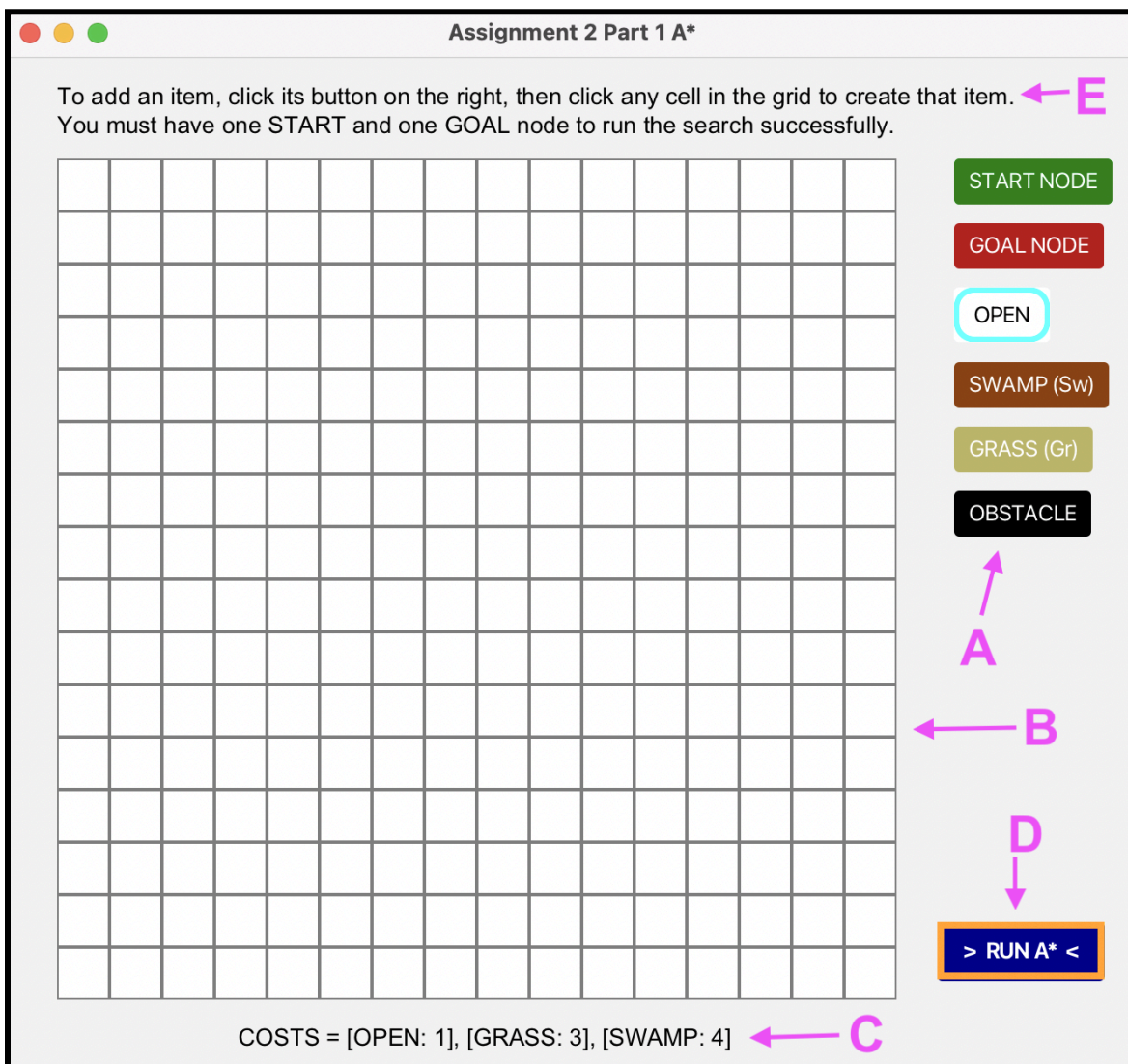


IMAGE 3: Initial Grid Screen

PART 4: Current Game Bugs/ Problems

- PROBLEMS/LIMITATIONS:
 - The ant animation is simplistic and could have an animated moving character added in.
 - If the user wants to reuse a grid design, they will have to recreate it from scratch, which does hinder game play and time.
 - If you want to fill a large area with a specific terrain, you have to click each individual node, which can be time consuming.
 - I did not add a restart button as it was not fully resetting the scene and I was concerned it would impede gameplay.
 - I did not implement any audio, which could enhance gameplay if the game was to be built further.
- BUGS:
 - I have not run into any known crashes or performance issues when testing gameplay.